Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Programmation Parallèle — TP2 — Hands-on MPI – Addition de Vecteurs, Matrice × Vecteur, Matrice × Matrice
4e année Cybersécurité - École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
KABORE Mohammad Sharif Jonathan

Filière : Cybersécurité

Année : 2026/2027

Enseignant : M. BAKHOUYA

Date : 11 octobre 2026

Table des matières

Rappel des objectifs du TP **2**

Architecture du projet **2**

Protocole de mesure et validation **3**

1 Exercice 1 : Addition de vecteurs **4**
1.1 Principe 4
1.2 Résultats 4

2 Exercice 2 : Multiplication matrice × vecteur **5**
2.1 Principe 5
2.2 Résultats 5

3 Exercice 3 : Multiplication matrice × matrice **6**
3.1 Principe 6
3.2 Résultats 7

4 Étude de complexité séquentielle **8**

5 Synthèse et interprétation **9**

Conclusion **10**

Rappel des objectifs du TP

Ce TP a pour objectif d'utiliser **MPI** pour paralléliser des opérations sur des vecteurs et des matrices, puis d'**analyser les performances** obtenues. Trois opérations sont implémentées :

- **Exercice 1 : Addition de deux vecteurs** $c = a + b$ (répartition par `MPI_Scatter`, calcul local, rassemblement par `MPI_Gather`)
- **Exercice 2 : Multiplication matrice** $\times$ **vecteur** $y = A \cdot x$ (diffusion de x par `MPI_Bcast`, bandes de lignes de A distribuées par `MPI_Scatter`)
- **Exercice 3 : Multiplication matrice** $\times$ **matrice** $C = A \cdot B$ (diffusion de B, bandes de lignes de A distribuées, rassemblement des bandes de C)

Pour chaque opération, les temps d'exécution sont mesurés en fonction de la **taille des données** et du **nombre de processus** $p \in \{1, 2, 4, 8\}$, afin de comparer les performances et d'analyser l'influence de la distribution des données et du parallélisme. Les résultats sont confrontés à un modèle simple $T_{par} \approx T_{calc}/p + T_{comm}$.

Architecture du projet

Le code fourni par l'enseignant (`VectAdd.c` et `MatMulti.c`) est conservé (logique, noms de variables, appels MPI) et réorganisé en modules. Les ajouts sont les chronomètres, la vérification du résultat, les versions séquentielles de référence, l'exercice 2 (écrit sur le même moule que `MatMulti.c`) et le benchmark :

- `vectadd.c / vectadd.h` — Exercice 1 : addition de vecteurs
- `matvec.c / matvec.h` — Exercice 2 : matrice $\times$ vecteur
- `matmul.c / matmul.h` — Exercice 3 : matrice $\times$ matrice
- `benchmark.c / benchmark.h` — mesures, vérification, jeux de tests
- `outils.c / outils.h` — chronomètre (maximum sur les processus) et écriture du CSV
- `main.c` — initialisation MPI et aiguillage vers le test ou le benchmark
- `Makefile` — compilation, tests (`make test`), mesures (`make bench`) et graphes (`make graphes`)
- `graphes.py` — génération des 15 graphes (`matplotlib`) à partir de `resultats.csv`

Comme dans le code du prof, `SIZE` et `ARRAY_SIZE` sont des constantes de compilation : le Makefile produit un exécutable par taille (`-DSIZE=n`, `-DARRAY_SIZE=n`) et lance chacun avec $p \in \{1, 2, 4, 8\}$. Les tailles doivent être divisibles par p (restriction du code du prof, vérifiée dans `main.c`). Les tableaux sont des `int` ; les débordements de la matrice 1024×1024 sont tolérés (`-fwrapv`) et n'affectent ni les temps ni la vérification, séquentiel et parallèle bouclant de la même façon.

Protocole de mesure et validation

Pour chaque opération, chaque taille et chaque p, la version parallèle est exécutée **3 fois** et la **moyenne** est retenue. Les grandeurs relevées sont :

- T_{par} : temps entre deux `MPI_Barrier`, incluant la distribution, le calcul et le rassemblement
- T_{calc} : maximum, sur les processus, du temps de calcul local (`MPI_Reduce` avec `MPI_MAX`)
- $T_{comm} = T_{par} - T_{calc}$: communication et synchronisation
- T_{seq} : temps de la version séquentielle, mesuré par le rang 0 sur les mêmes données
- Accélération $S = T_{seq}/T_{par}$ et efficacité $E = S/p$

Le rang 0 compare le résultat parallèle au résultat séquentiel (`OK`/`ECHEC`). La cible `make test` exécute les trois programmes avec `SIZE` $= 8$ pour $p \in \{1, 2, 4, 8\}$, avec l'affichage des matrices du prof.

Complexité théorique

Opération	**Calcul séq.**	**Calcul / processus**	**Données échangées**	**Calcul / communication**
Addition de vecteurs	$O(n)$	$O(n/p)$	$\approx 3n$	$O(1)$
Matrice × vecteur	$O(n^2)$	$O(n^2/p)$	$\approx n^2$	$O(1)$
Matrice × matrice	$O(n^3)$	$O(n^3/p)$	$\approx 3n^2$	$O(n)$

TABLE 1 – Complexité des trois opérations (n : taille du vecteur ou de la matrice carrée)

Le maître envoie et reçoit toujours la totalité des données quel que soit p : T_{comm} **ne diminue pas avec** p. L'accélération est donc bornée par $S \leq T_{seq}/(T_{seq}/p + T_{comm})$. Elle ne peut être intéressante que si le rapport calcul/communication est grand, ce qui n'est le cas que pour le produit matrice × matrice.

1 Exercice 1 : Addition de vecteurs

1.1 Principe

Le maître (rang 0) initialise a et b, distribue à chaque processus un bloc de n/p éléments de chacun par `MPI_Scatter`, chaque processus additionne ses blocs, puis le maître rassemble les résultats par `MPI_Gather`.

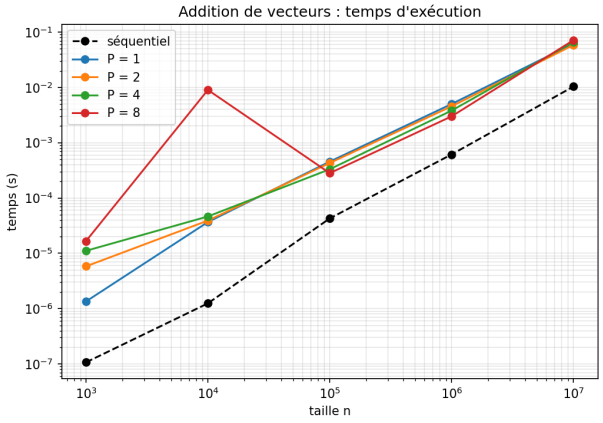
```
MPI_Scatter(a, n_per_proc, MPI_INT, ap, n_per_proc, MPI_INT, MASTER,
    MPI_COMM_WORLD);
MPI_Scatter(b, n_per_proc, MPI_INT, bp, n_per_proc, MPI_INT, MASTER,
    MPI_COMM_WORLD);

for (i = 0; i < n_per_proc; i++) {
    cp[i] = ap[i] + bp[i];
}

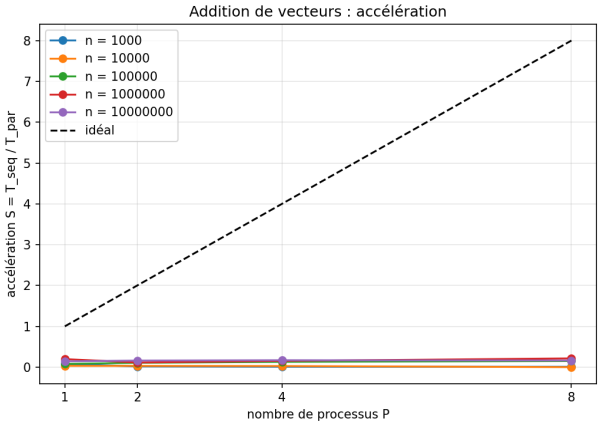
MPI_Gather(cp, n_per_proc, MPI_INT, c, n_per_proc, MPI_INT, MASTER,
    MPI_COMM_WORLD);
```

1.2 Résultats

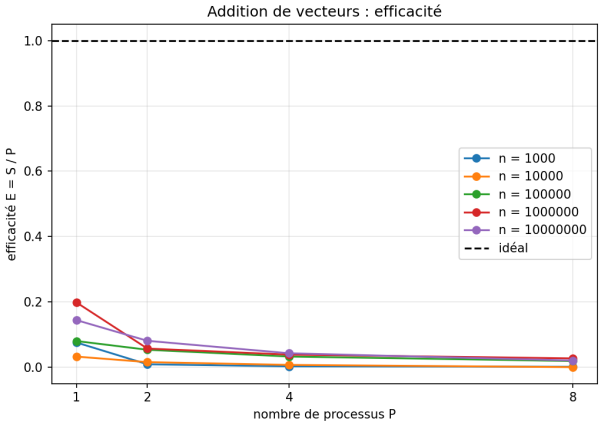
Tailles testées : $n \in \{10^3, 10^4, 10^5, 10^6, 10^7\}$.



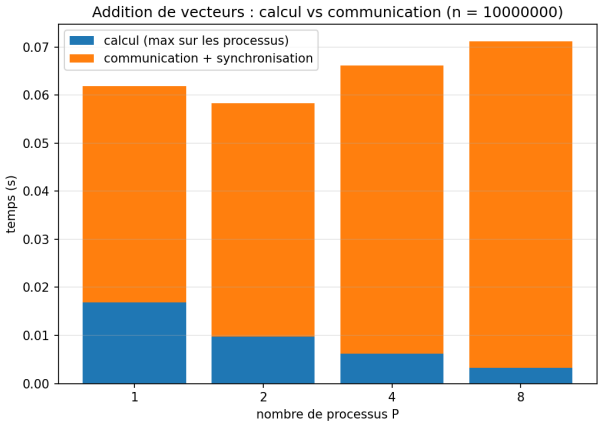
(a) Temps d'exécution



(b) Accélération



(c) Efficacité



(d) Calcul vs communication ($n = 10^7$)

FIGURE 1 – Addition de vecteurs : performances en fonction de n et de p

Observations :

— L'accélération reste **inférieure à 0,2** pour toutes les tailles et tous les p (figure 1b) : la version parallèle est **plus lente** que la version séquentielle, et l'efficacité (figure 1c) s'effondre quand p augmente.

— Même avec $p = 1$, la version MPI est environ 5 à 7 fois plus lente (efficacité $\approx 0{,}2$ pour $n = 10^6$ et $\approx 0{,}14$ pour $n = 10^7$) : le `Scatter`/`Gather` impose des copies mémoire et des premières écritures de pages que la boucle séquentielle, qui travaille en place, n'a pas.

— Pour $n = 10^7$ (figure 1d), le temps de calcul diminue bien, d'environ 0,017 s ($p = 1$) à 0,003 s ($p = 8$), mais la communication reste de l'ordre de 0,045 à 0,07 s : à $p = 8$, le calcul représente moins de 5 % du temps total.

— Les courbes parallèles de la figure 1a sont quasi confondues et restent au-dessus du séquentiel. Le pic à $p = 8$, $n = 10^4$ (≈ 9 ms contre quelques dizaines de μs pour les autres p) est attribué à un aléa de mesure (moyenne sur seulement 3 répétitions, processus en concurrence pour les cœurs).

2 Exercice 2 : Multiplication matrice × vecteur

2.1 Principe

Même schéma que `MatMulti.c` : le vecteur x est diffusé à tous les processus (`MPI_Bcast`), chaque processus reçoit une bande de n/p lignes de A (`MPI_Scatter`), calcule les composantes de y correspondantes, puis le maître les rassemble (`MPI_Gather`).

```
from = myrank * SIZE/P;
to = (myrank+1) * SIZE/P;

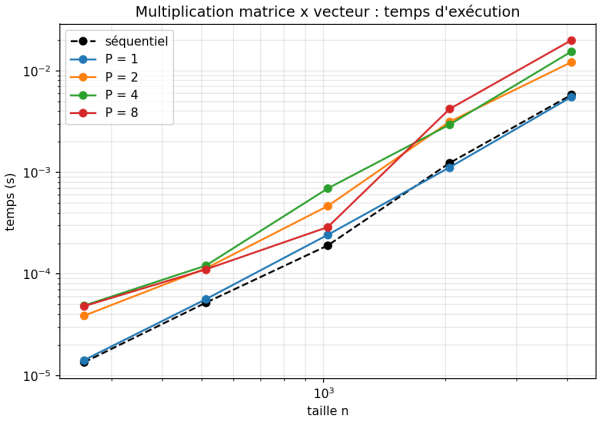
MPI_Bcast(x, SIZE, MPI_INT, 0, MPI_COMM_WORLD);
MPI_Scatter(A, SIZE*SIZE/P, MPI_INT, A[from], SIZE*SIZE/P, MPI_INT, 0,
    MPI_COMM_WORLD);

for (i = from; i < to; i++) {
    y[i] = 0;
    for (k = 0; k < SIZE; k++) {
        y[i] += A[i][k] * x[k];
    }
}

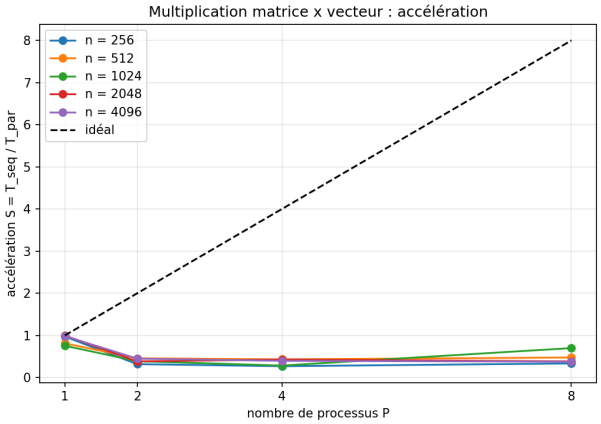
MPI_Gather(&y[from], SIZE/P, MPI_INT, y, SIZE/P, MPI_INT, 0,
   MPI_COMM_WORLD);
```

2.2 Résultats

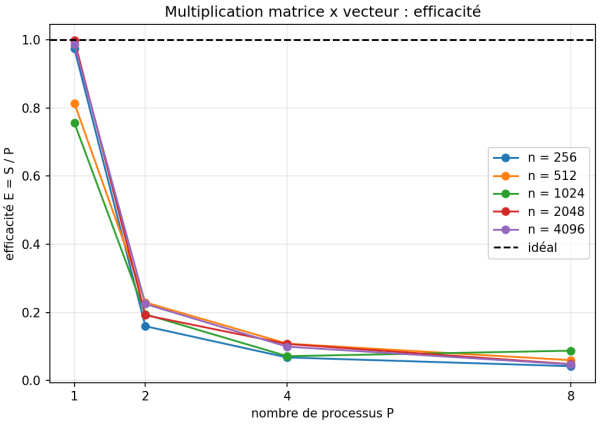
Tailles testées : $n \in \{256, 512, 1024, 2048, 4096\}$.



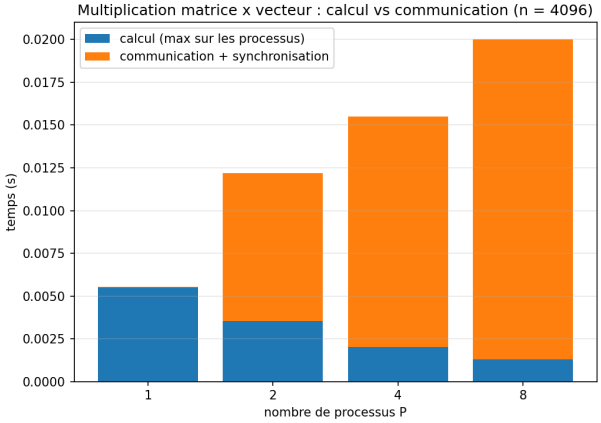
(a) Temps d'exécution



(b) Accélération



(c) Efficacité



(d) Calcul vs communication ($n = 4096$)

FIGURE 2 – Matrice × vecteur : performances en fonction de n et de p

Observations :

— Pour toutes les tailles, l'accélération reste **inférieure ou égale à 1** (figure 2b) : environ 0,3 à 0,45 pour $p = 2$ et $p = 4$, et au mieux 0,7 pour $p = 8$ ($n = 1024$). L'efficacité est inférieure à 0,25 dès $p = 2$ et de l'ordre de 0,05 à 0,09 pour $p = 8$.

— Sur la figure 2a, la courbe $p = 1$ suit le séquentiel, tandis que toutes les courbes $p \geq 2$ restent au-dessus : la parallélisation **ralentit** le calcul pour toutes les tailles testées.

— Pour $n = 4096$ (figure 2d), le temps de calcul passe de $\approx 5{,}5$ ms ($p = 1$) à $\approx 1{,}3$ ms ($p = 8$), mais la communication passe de ≈ 9 ms ($p = 2$) à ≈ 19 ms ($p = 8$) : le temps total est multiplié par $\approx 3{,}6$ entre $p = 1$ et $p = 8$.

— Explication : chaque entier de A est transféré une seule fois pour n'effectuer qu'une multiplication-addition. Le rapport calcul/communication est $O(1)$ (tableau de complexité) et ne s'améliore pas avec n.

3 Exercice 3 : Multiplication matrice × matrice

3.1 Principe

Code de `MatMulti.c` conservé : la matrice B est diffusée à tous les processus, chaque processus reçoit une bande de n/p lignes de A, calcule la bande correspondante de C, et le maître rassemble les bandes.

```
from = myrank * SIZE/P;
to = (myrank+1) * SIZE/P;

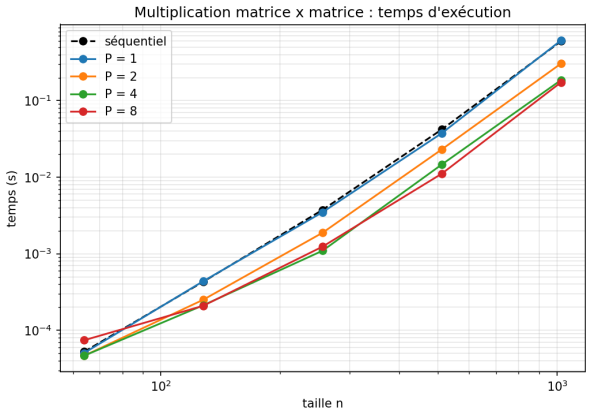
MPI_Bcast(B, SIZE*SIZE, MPI_INT, 0, MPI_COMM_WORLD);
MPI_Scatter(A, SIZE*SIZE/P, MPI_INT, A[from], SIZE*SIZE/P, MPI_INT, 0,
    MPI_COMM_WORLD);

for (i = from; i < to; i++) {
    for (j = 0; j < SIZE; j++) {
        C[i][j] = 0;
        for (k = 0; k < SIZE; k++) {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}

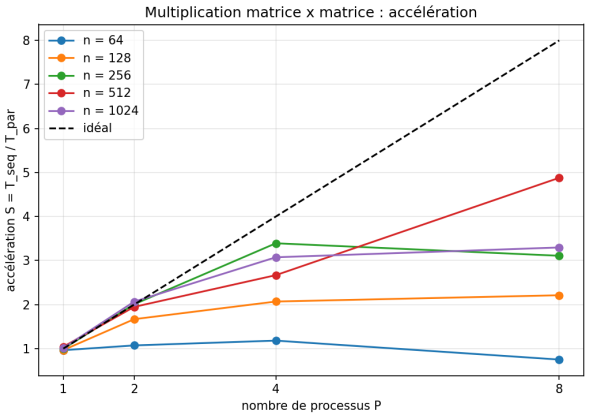
MPI_Gather(C[from], SIZE*SIZE/P, MPI_INT, C, SIZE*SIZE/P, MPI_INT, 0,
   MPI_COMM_WORLD);
```

3.2 Résultats

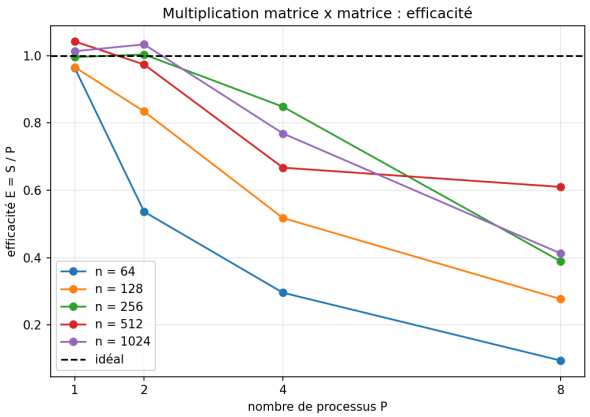
Tailles testées : $n \in \{64, 128, 256, 512, 1024\}$.



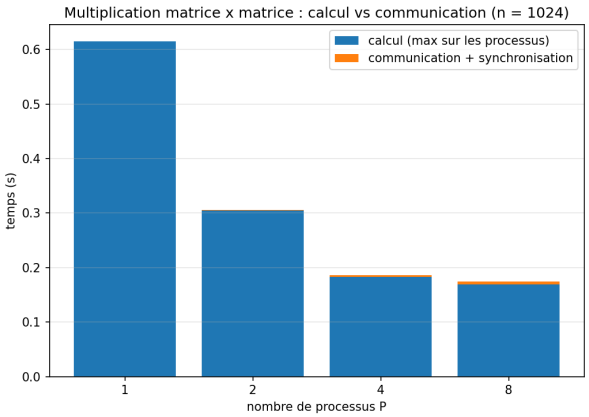
(a) Temps d'exécution



(b) Accélération



(c) Efficacité



(d) Calcul vs communication ($n = 1024$)

FIGURE 3 – Matrice × matrice : performances en fonction de n et de p

Observations :

— C'est la seule opération où la parallélisation est **rentable** (figure 3b). Pour $n = 1024$, l'accélération vaut $\approx 2{,}1$ ($p = 2$), $\approx 3{,}1$ ($p = 4$) et $\approx 3{,}3$ ($p = 8$). Le maximum observé est $\approx 4{,}9$ ($n = 512$, $p = 8$).

— L'efficacité (figure 3c) est voisine de 1 pour $p = 2$ dès $n \geq 256$ (les valeurs légèrement supérieures à 1 sont dues à des aléas de mesure et à des effets de cache), de 0,67 à 0,85 pour $p = 4$ et de 0,39 à 0,61 pour $p = 8$. Elle est d'autant meilleure que n est grand, conformément au rapport calcul/communication en $O(n)$.

— Pour $n = 64$, la parallélisation ne rapporte rien (accélération $\leq 1{,}2$, et $\approx 0{,}75$ pour $p = 8$) : le calcul séquentiel dure environ 50 μs, ce qui ne suffit pas à amortir le coût de la distribution.

— Pour $n = 1024$ (figure 3d), le temps passe de $\approx 0{,}615$ s ($p = 1$) à $\approx 0{,}305$ s, 0,185 s puis 0,174 s : la communication est à peine visible (de l'ordre de 1 à 3 % du temps total).

— Le gain s'essouffle entre $p = 4$ et $p = 8$: le temps de calcul maximal ne passe que de $\approx 0{,}183$ s à $\approx 0{,}168$ s au lieu d'être divisé par 2. Les explications possibles sont un nombre de cœurs physiques inférieur à 8 (processus en concurrence) et la saturation de la bande passante mémoire (chaque processus lit B en entier). L'irrégularité de $n = 512$ (meilleur point à $p = 8$) peut venir des effets de cache et des 3 répétitions seulement.

4 Étude de complexité séquentielle

Les figures suivantes montrent T_{seq} en échelle log-log (à gauche, avec la pente mesurée et la référence théorique) et normalisé par n^k (à droite) ; ce rapport doit être constant si le modèle de complexité tient.

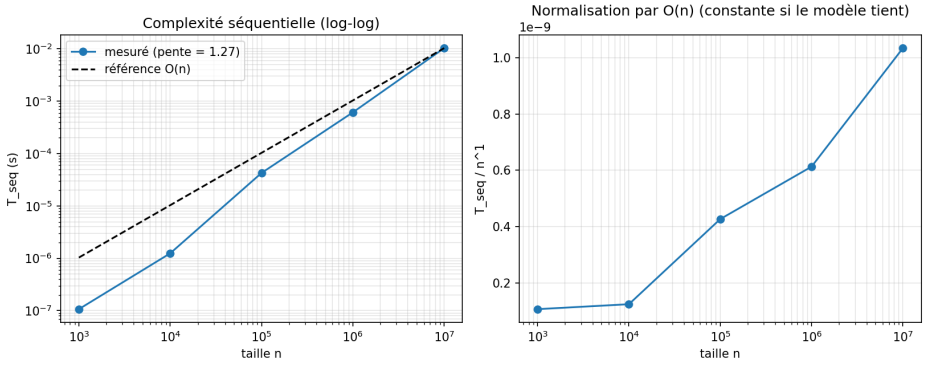


FIGURE 4 – Addition de vecteurs : pente mesurée 1,27 (théorie : 1)

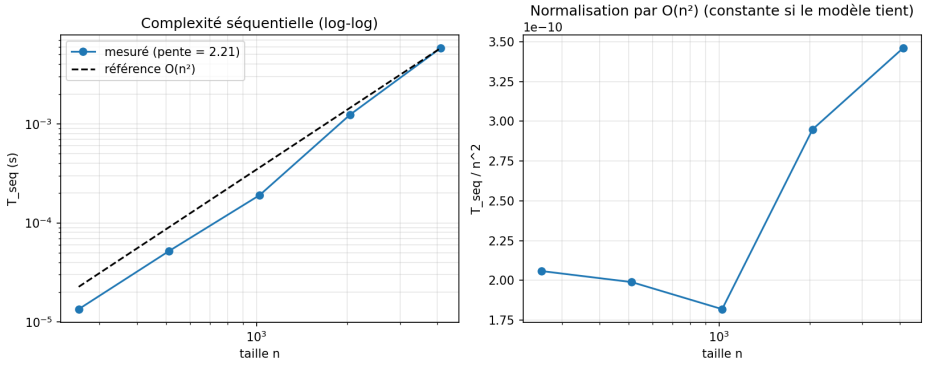


FIGURE 5 – Matrice × vecteur : pente mesurée 2,21 (théorie : 2)

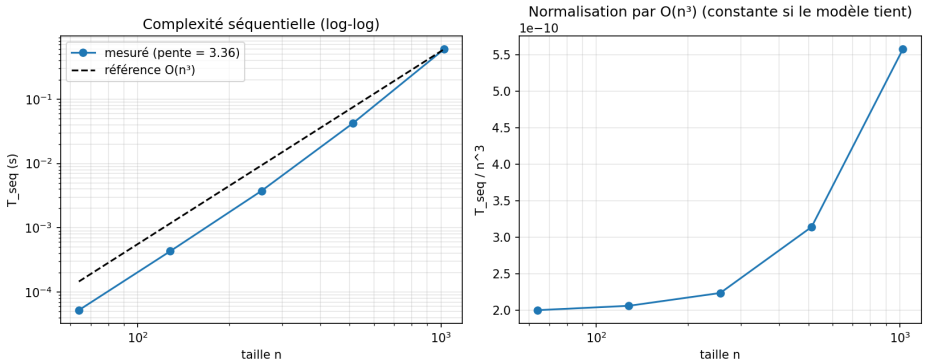


FIGURE 6 – Matrice × matrice : pente mesurée 3,36 (théorie : 3)

Observations :

— Les trois pentes sont supérieures à l'exposant théorique, et le rapport T_{seq}/n^k n'est pas constant mais **croît avec** n. Ce n'est pas une remise en cause de la complexité algorithmique, mais un effet de **hiérarchie mémoire** : le coût par opération augmente quand les données ne tiennent plus dans le cache.

— Addition de vecteurs : le temps par élément passe de $\approx 10^{-10}$ s ($n = 10^3$) à $\approx 10^{-9}$ s ($n = 10^7$), où les trois tableaux de 40 Mo chacun sont limités par la mémoire.

— Matrice × vecteur : T_{seq}/n^2 reste voisin de 2×10^{-10} s jusqu'à $n = 1024$, puis passe à $\approx 3 \times 10^{-10}$ s pour $n = 2048$ (16 Mo) et $n = 4096$ (64 Mo).

— Matrice × matrice : T_{seq}/n^3 passe de 2×10^{-10} s ($n = 64$) à $\approx 5{,}6 \times 10^{-10}$ s ($n = 1024$). La boucle i, j, k du prof parcourt B colonne par colonne (accès non contigus), ce qui devient coûteux dès que B (4 Mo) dépasse les caches.

5 Synthèse et interprétation

Opération	**Calcul / comm.**	**Accélération max observée**	**Verdict**
Addition de vecteurs	$O(1)$	$< 0{,}2$	Parallélisation non rentable à toutes les tailles
Matrice × vecteur	$O(1)$	≤ 1 ($\approx 0{,}7$ pour $p \geq 2$)	Parallélisation non rentable à toutes les tailles
Matrice × matrice	$O(n)$	$\approx 4{,}9$ ($n = 512$, $p = 8$)	Rentable dès $n \geq 128$, de mieux en mieux avec n

TABLE 2 – Synthèse des performances (valeurs relevées sur les graphes)

Le modèle $T_{par} \approx T_{calc}/p + T_{comm}$ explique l'ensemble des résultats :

— **Distribution des données** : dans les trois cas, le maître distribue et rassemble tout, et T_{comm} ne diminue pas avec p (il augmente même légèrement pour matrice × vecteur). La communication est pourtant une copie en mémoire partagée puisque tous les processus tournent sur la même machine, et elle coûte déjà plus que le calcul quand celui-ci est léger.

— **Intensité de calcul** : seule la multiplication de matrices effectue $O(n)$ opérations par donnée transférée. C'est la seule à amortir la communication et à voir son efficacité s'améliorer quand n augmente.
— **Limites matérielles** : au-delà de $p = 4$, l'efficacité chute même pour matrice × matrice (cœurs disponibles et bande passante mémoire), ce qui limite l'accélération à environ 3 à 5 pour $p = 8$.

Conclusion

Ce TP a permis d'implémenter avec MPI trois opérations classiques (addition de vecteurs, produit matrice × vecteur, produit matrice × matrice) selon le schéma du code fourni (`Bcast`, `Scatter`, calcul local, `Gather`) et d'analyser leurs performances.

Résultats clés :
— L'**addition de vecteurs** et le produit **matrice × vecteur** sont dominés par la communication : la version parallèle est plus lente que la version séquentielle, quelles que soient la taille et le nombre de processus.
— Le produit **matrice × matrice** bénéficie du parallélisme (accélération jusqu'à $\approx 4{,}9$, efficacité proche de 1 pour $p = 2$) car son calcul en $O(n^3)$ domine une communication en $O(n^2)$.
— Les écarts aux complexités théoriques observés en séquentiel (1,27, 2,21 et 3,36 au lieu de 1, 2 et 3) s'expliquent par les effets de cache.

Limites et pistes : les mesures reposent sur 3 répétitions seulement (un point aberrant, comme le pic de l'addition de vecteurs à $p = 8$, pèse sur la moyenne) et sur une seule machine à mémoire partagée. Les pistes d'amélioration sont l'usage de la médiane sur davantage de répétitions, un ordre de boucles i, k, j dans le produit matriciel (accès contigus à B) et, pour les opérations légères, un chronométrage qui exclut la distribution lorsque les données sont déjà réparties.